

Reviewer Pack — Minimal Order of Birational Geometry and Communication Compl...

Entry 75b4a63dbfab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 20:14:04 UTC

Minimal Order of Birational Geometry and Communication Complexity Rank Correlation

Entry ID: 75b4a63dbfab **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 20:14:04 UTC

1. Conjecture

Field A (mathematical branch): Birational Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every birational morphism φ from a smooth projective curve C to the projective line P^1 , the minimal number of points on a general fiber of φ is linearly correlated with its communication complexity rank $r(\varphi)$, such that $\log_2(n^{r(\varphi)+1}) \leq w(\varphi) + H(\varphi)$ for some constants n and $H(\varphi)$.

Rationale (proposer's reasoning):

Birational geometry provides a geometric framework to study the structure of complex systems, and its minimal point counts could reveal intrinsic properties related to communication complexity. This conjecture aims to expose hidden connections between geometric invariants and algorithmic complexities.

Taxonomy category: Birational_Geometry_to_Communication_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f6f26595093f9a58

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Evaluate birational morphisms φ from smooth projective curves C to P^1 . If $\log_2(n^{r(\varphi)+1}) \leq w(\varphi) + H(\varphi)$, with a correlation coefficient ≥ 0.7 over 30 seeds, support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "birational geometry" AND "communication complexity" AND "matrix rank" - "minimal order" AND "general fiber points" AND "communication complexity rank" - "log₂(n^{r(φ)+1})" AND "w(φ) + H(φ)" AND "birational morphism"

Top relevant hits considered: - [s2:10.1007/jhep07(2025)086] Differential cross-section measurements of D[±] and

$$D_s^\pm$$

meson production in proton-proton collisions at \$\$ - [s2:10.1103/PhysRevLett.125.152001] Observation of the B_s⁰→X(3872)φ Decay.

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_curve(n):
        # Generate a random smooth projective curve with n variables
        return [random.randint(1, 2*n) for _ in range(n)]

    def birational_morphism(curve):
        # Simulate a birational morphism from C to P^1
        return sum(curve)

    def communication_complexity_rank(morphism):
        # Simulate the rank of communication complexity using a small DPLL solver or other efficient
        # For simplicity, we use a placeholder function that returns a random integer
        return random.randint(1, 5)

    def entropy(p):
        if p == 0 or p == 1:
            return 0
        return -p * math.log2(p) - (1 - p) * math.log2(1 - p)

    n = random.choice([5, 10, 15, 20, 30, 40])
    curve = generate_curve(n)
```

```

morphism = birational_morphism(curve)
r_phi = communication_complexity_rank(morphism)

log_value = math.log2(n**(r_phi + 1))
w_phi = Fraction(morphism) / n
H_phi = entropy(Fraction(morphism) / n)

metric_value = log_value <= w_phi + H_phi

return {
    "metric_name": "log_value <= w_phi + H_phi",
    "metric_value": float(metric_value),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": metric_value,
    "counterexample": ""
}

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        # Default list of 30 primes
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7dbf1f4.py", line 68, in <module>
    trial_result = run_trial(seed)
    ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7dbf1f4.py", line 46, in run_trial

```

```
H_phi = entropy(Fraction(morphism) / n)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7dbf1f4.py", line 37, in entropy
    return -p * math.log2(p) - (1 - p) * math.log2(1 - p)
    ^^^^^^^^^^^^^^^^^^^^^
```

ValueError: math domain error

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a math domain error, preventing the evaluation of the conjecture's conditions. | next: Review and debug the test code to ensure it can complete without errors before re-evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14886
2	preregistration	ollama_remote	glm4:latest	0	0	9982
3	novelty	ollama_remote	glm4:latest	0	0	8546
4	novelty	ollama_remote	glm4:latest	0	0	9752
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15382
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11567
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8038
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9412
9	judge	ollama_remote	glm4:latest	0	0	12335

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99900 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/75b4a63dbfab.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/75b4a63dbfab.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/75b4a63dbfab.tar.gz* (if generated)